

451. Sort Characters By Frequency

Medium Topics Companies

Given a string `s`, sort it in **decreasing order** based on the **frequency** of the characters. The **frequency** of a character is the number of times it appears in the string.

Return the *sorted string*. If there are multiple answers, return *any of them*.

Example 1:

Input: `s = "tree"`
Output: `"eert"`
Explanation: 'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:

Input: `s = "cccaaa"`
Output: `"aaaccc"`
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers. Note that "cacaca" is incorrect, as the same characters must be together.

Example 3:

Input: `s = "Aabb"`
Output: `"bbAa"`
Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect. Note that 'A' and 'a' are treated as two different characters.

Constraints:

- $1 \leq s.length \leq 5 \times 10^5$
- `s` consists of uppercase and lowercase English letters and digits.

Code

Java Auto

```
1 import java.util.*;
2
3 class Solution {
4     public String frequencySort(String s) {
5
6         // Count frequency of each character
7         Map<Character, Integer> freq = new HashMap<>();
8
9         for (char c : s.toCharArray()) {
10             freq.put(c, freq.getOrDefault(c, 0) + 1);
11         }
12
13         // Sort characters by decreasing frequency
14         List<Character> chars = new ArrayList<>(freq.keySet());
15
16         chars.sort((a, b) -> freq.get(b) - freq.get(a));
17     }
18 }
```

Saved

Ln 28, Col 1

Testcase Test Result

Accepted Runtime: 1 ms

Case 1 Case 2 Case 3

Input

`s = "tree"`

Output

`"eert"`

Expected

`"eert"`

Contribute a testcase